BookMyShow
Ticketing Backend — Database Design

P1: Schema · Normalization · Concurrency Strategy
P2: Show Listing Query

MySQL • InnoDB • Row-level Locking • Idempotent Webhooks

1. Entities and Attributes

users

Platform users who make bookings.

Column	Type	Constraints
id	BIGINT UNSIGNED	PK, AUTO_INCREMENT
name	VARCHAR(120)	NOT NULL
email	VARCHAR(255)	NOT NULL, UNIQUE
phone	VARCHAR(15)	NOT NULL, UNIQUE
created_at	DATETIME	NOT NULL, DEFAULT NOW()

movies

Movie catalogue — language, genre, duration.

Column	Type	Constraints
id	BIGINT UNSIGNED	PK, AUTO_INCREMENT
title	VARCHAR(255)	NOT NULL
duration_mins	SMALLINT UNSIGNED	NOT NULL
language	VARCHAR(50)	NOT NULL
genre	VARCHAR(80)	NOT NULL
rating	CHAR(5)	nullable (e.g. U/A, A)
release_date	DATE	NOT NULL

theatres

Physical cinema locations.

Column	Type	Constraints
id	BIGINT UNSIGNED	PK, AUTO_INCREMENT
name	VARCHAR(255)	NOT NULL
city	VARCHAR(100)	NOT NULL
address	TEXT	NOT NULL

screens

Auditoriums inside a theatre.

Column	Type	Constraints
id	BIGINT UNSIGNED	PK, AUTO_INCREMENT
theatre_id	BIGINT UNSIGNED	FK → theatres.id
name	VARCHAR(60)	NOT NULL
total_seats	SMALLINT UNSIGNED	NOT NULL
–	–	UNIQUE (theatre_id, name)

seats

Fixed physical seats in a screen. Created once, never deleted.

Column	Type	Constraints
id	BIGINT UNSIGNED	PK, AUTO_INCREMENT
screen_id	BIGINT UNSIGNED	FK → screens.id
row_label	CHAR(3)	NOT NULL (e.g. A, B, AA)
seat_number	TINYINT UNSIGNED	NOT NULL, 1-based
seat_type	ENUM	REGULAR / PREMIUM / RECLINER
—	—	UNIQUE (screen_id, row_label, seat_number)

shows

One screening of a movie on a specific screen at a date/time.

Column	Type	Constraints
id	BIGINT UNSIGNED	PK, AUTO_INCREMENT
movie_id	BIGINT UNSIGNED	FK → movies.id
screen_id	BIGINT UNSIGNED	FK → screens.id
show_date	DATE	NOT NULL
start_time	TIME	NOT NULL
base_price	DECIMAL(8,2)	NOT NULL
—	—	UNIQUE (screen_id, show_date, start_time)

show_seats

Per-show availability state for every seat. Core of concurrency control.

Column	Type	Constraints
id	BIGINT UNSIGNED	PK, AUTO_INCREMENT
show_id	BIGINT UNSIGNED	FK → shows.id
seat_id	BIGINT UNSIGNED	FK → seats.id
status	ENUM	AVAILABLE / HELD / BOOKED
hold_expires_at	DATETIME	nullable — set when status=HELD
booking_id	BIGINT UNSIGNED	FK → bookings.id, nullable
version	INT UNSIGNED	optimistic lock counter
—	—	UNIQUE (show_id, seat_id)

bookings

A user's purchase attempt for one or more seats at a show.

Column	Type	Constraints
id	BIGINT UNSIGNED	PK, AUTO_INCREMENT
user_id	BIGINT UNSIGNED	FK → users.id
show_id	BIGINT UNSIGNED	FK → shows.id
status	ENUM	PENDING / CONFIRMED / CANCELLED / FAILED
total_amount	DECIMAL(10,2)	NOT NULL
created_at	DATETIME	NOT NULL
updated_at	DATETIME	NOT NULL, ON UPDATE CURRENT_TIMESTAMP

payments

Payment records. Idempotency enforced via UNIQUE external_txn_id.

Column	Type	Constraints
id	BIGINT UNSIGNED	PK, AUTO_INCREMENT
booking_id	BIGINT UNSIGNED	FK → bookings.id
amount	DECIMAL(10,2)	NOT NULL
status	ENUM	PENDING / SUCCESS / FAILED / REFUNDED
gateway	VARCHAR(60)	NOT NULL (e.g. razorpay, stripe)
external_txn_id	VARCHAR(255)	NOT NULL, UNIQUE
payload	JSON	nullable — raw webhook body for audit
created_at	DATETIME	NOT NULL
updated_at	DATETIME	NOT NULL, ON UPDATE

2. Sample Rows

users

id	name	email	phone
1	Arjun Mehta	arjun@example.com	9876543210
2	Priya Sharma	priya@example.com	9123456780
3	Rohan Kapoor	rohan@example.com	9988776655

movies

id	title	duration	language	genre	rating
1	Kalki 2898 AD	180 min	Telugu	Sci-Fi / Action	U/A
2	Stree 2	135 min	Hindi	Horror Comedy	U/A
3	Devara: Part 1	166 min	Telugu	Action Drama	U/A

theatres

id	name	city
1	PVR Cinemas – Phoenix Mall	Mumbai
2	INOX – Lulu Mall	Kochi
3	Cinepolis – VR Chennai	Chennai

screens

id	theatre_id	name	total_seats
1	1	Screen 1	200
2	1	Screen 2	150
3	1	Screen 3	120
4	2	Audi A	180
5	2	Audi B	160

shows (PVR Mumbai, 2024-10-01)

id	movie	screen	show_date	start_time	base_price
1	Kalki 2898 AD	Screen 1	2024-10-01	10:00	■250
2	Kalki 2898 AD	Screen 1	2024-10-01	14:00	■300
3	Stree 2	Screen 2	2024-10-01	11:30	■200
4	Devara: Part 1	Screen 3	2024-10-01	18:00	■350

show_seats (show_id = 1)

show_id	seat_id	status	hold_expires_at	booking_id
1	1	BOOKED	NULL	1

show_id	seat_id	status	hold_expires_at	booking_id
1	2	BOOKED	NULL	1
1	3	AVAILABLE	NULL	NULL
1	4	HELD	2024-10-01 10:15:00	NULL

bookings

id	user_id	show_id	status	total_amount
1	1	1	CONFIRMED	500.00

payments

id	booking_id	amount	status	gateway	external_txn_id
1	1	500.00	SUCCESS	razorpay	rzp_live_abc123XYZ

3. P1 — CREATE TABLE SQL (schema.sql)

All statements are directly executable on MySQL 8.0+. Run schema.sql first, then sample_data.sql.

```
-- BookMyShow Ticketing Backend — MySQL Schema

CREATE TABLE users (
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(120) NOT NULL,
email VARCHAR(255) NOT NULL,
phone VARCHAR(15) NOT NULL,
created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
UNIQUE KEY uq_users_email (email),
UNIQUE KEY uq_users_phone (phone)
);

CREATE TABLE movies (
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
title VARCHAR(255) NOT NULL,
duration_mins SMALLINT UNSIGNED NOT NULL,
language VARCHAR(50) NOT NULL,
genre VARCHAR(80) NOT NULL,
rating CHAR(5),
release_date DATE NOT NULL
);

CREATE TABLE theatres (
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(255) NOT NULL,
city VARCHAR(100) NOT NULL,
address TEXT NOT NULL
);

CREATE TABLE screens (
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
theatre_id BIGINT UNSIGNED NOT NULL,
name VARCHAR(60) NOT NULL,
total_seats SMALLINT UNSIGNED NOT NULL,
CONSTRAINT fk_screens_theatre FOREIGN KEY (theatre_id)
REFERENCES theatres (id) ON DELETE CASCADE,
UNIQUE KEY uq_screen_name_per_theatre (theatre_id, name)
);

CREATE TABLE seats (
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
screen_id BIGINT UNSIGNED NOT NULL,
row_label CHAR(3) NOT NULL,
seat_number TINYINT UNSIGNED NOT NULL,
seat_type ENUM('REGULAR','PREMIUM','RECLINER')
NOT NULL DEFAULT 'REGULAR',
CONSTRAINT fk_seats_screen FOREIGN KEY (screen_id)
```

```
REFERENCES screens (id) ON DELETE CASCADE,
UNIQUE KEY uq_seat_position (screen_id, row_label, seat_number)
);

CREATE TABLE shows (
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
movie_id BIGINT UNSIGNED NOT NULL,
screen_id BIGINT UNSIGNED NOT NULL,
show_date DATE NOT NULL,
start_time TIME NOT NULL,
base_price DECIMAL(8,2) NOT NULL,
CONSTRAINT fk_shows_movie FOREIGN KEY (movie_id)
REFERENCES movies (id),
CONSTRAINT fk_shows_screen FOREIGN KEY (screen_id)
REFERENCES screens (id),
UNIQUE KEY uq_show_slot (screen_id, show_date, start_time),
INDEX idx_shows_date_screen (show_date, screen_id)
);

CREATE TABLE show_seats (
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
show_id BIGINT UNSIGNED NOT NULL,
seat_id BIGINT UNSIGNED NOT NULL,
status ENUM('AVAILABLE','HELD','BOOKED')
NOT NULL DEFAULT 'AVAILABLE',
hold_expires_at DATETIME NULL,
booking_id BIGINT UNSIGNED NULL,
version INT UNSIGNED NOT NULL DEFAULT 0,
CONSTRAINT fk_ss_show FOREIGN KEY (show_id) REFERENCES shows (id),
CONSTRAINT fk_ss_seat FOREIGN KEY (seat_id) REFERENCES seats (id),
UNIQUE KEY uq_show_seat (show_id, seat_id),
INDEX idx_ss_show_status (show_id, status),
INDEX idx_ss_expires (hold_expires_at)
);

CREATE TABLE bookings (
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
user_id BIGINT UNSIGNED NOT NULL,
show_id BIGINT UNSIGNED NOT NULL,
status ENUM('PENDING','CONFIRMED','CANCELLED','FAILED')
NOT NULL DEFAULT 'PENDING',
total_amount DECIMAL(10,2) NOT NULL,
created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
ON UPDATE CURRENT_TIMESTAMP,
CONSTRAINT fk_bookings_user FOREIGN KEY (user_id)
REFERENCES users (id),
CONSTRAINT fk_bookings_show FOREIGN KEY (show_id)
REFERENCES shows (id),
INDEX idx_bookings_user (user_id),
INDEX idx_bookings_show (show_id)
```

```
);

ALTER TABLE show_seats
ADD CONSTRAINT fk_ss_booking FOREIGN KEY (booking_id)
REFERENCES bookings (id) ON DELETE SET NULL;

CREATE TABLE payments (
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
booking_id BIGINT UNSIGNED NOT NULL,
amount DECIMAL(10,2) NOT NULL,
status ENUM('PENDING','SUCCESS','FAILED','REFUNDED')
NOT NULL DEFAULT 'PENDING',
gateway VARCHAR(60) NOT NULL,
external_txn_id VARCHAR(255) NOT NULL,
payload JSON NULL,
created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
ON UPDATE CURRENT_TIMESTAMP,
CONSTRAINT fk_payments_booking FOREIGN KEY (booking_id)
REFERENCES bookings (id),
UNIQUE KEY uq_external_txn (external_txn_id)
);
```

4. P1 — Sample INSERT Statements (sample_data.sql)

```
-- users
INSERT INTO users (name, email, phone) VALUES
('Arjun Mehta', 'arjun@example.com', '9876543210'),
('Priya Sharma', 'priya@example.com', '9123456780'),
('Rohan Kapoor', 'rohan@example.com', '9988776655');

-- movies
INSERT INTO movies (title, duration_mins, language, genre, rating, release_date) VALUES
('Kalki 2898 AD', 180, 'Telugu', 'Sci-Fi / Action', 'U/A', '2024-06-27'),
('Stree 2', 135, 'Hindi', 'Horror Comedy', 'U/A', '2024-08-15'),
('Devara: Part 1', 166, 'Telugu', 'Action Drama', 'U/A', '2024-09-27');

-- theatres
INSERT INTO theatres (name, city, address) VALUES
('PVR Cinemas – Phoenix Mall', 'Mumbai', 'LBS Marg, Kurla West, Mumbai 400070'),
('INOX – Lulu Mall', 'Kochi', 'Lulu Mall, Edapally, Kochi 682024'),
('Cinepolis – VR Chennai', 'Chennai', 'VR Chennai, Koyambedu, Chennai 600092');

-- screens
INSERT INTO screens (theatre_id, name, total_seats) VALUES
(1,'Screen 1',200),(1,'Screen 2',150),(1,'Screen 3',120),
(2,'Audi A',180),(2,'Audi B',160),(3,'Hall 1',220);

-- seats (rows A & B, 5 seats each, in Screen 1)
INSERT INTO seats (screen_id, row_label, seat_number, seat_type) VALUES
(1,'A',1,'REGULAR'),(1,'A',2,'REGULAR'),(1,'A',3,'REGULAR'),
(1,'A',4,'REGULAR'),(1,'A',5,'REGULAR'),
(1,'B',1,'PREMIUM'),(1,'B',2,'PREMIUM'),(1,'B',3,'PREMIUM'),
(1,'B',4,'PREMIUM'),(1,'B',5,'PREMIUM');

-- shows
INSERT INTO shows (movie_id, screen_id, show_date, start_time, base_price) VALUES
(1, 1, '2024-10-01', '10:00:00', 250.00),
(1, 1, '2024-10-01', '14:00:00', 300.00),
(2, 2, '2024-10-01', '11:30:00', 200.00),
(3, 3, '2024-10-01', '18:00:00', 350.00);

-- show_seats for show 1
INSERT INTO show_seats (show_id, seat_id, status) VALUES
(1,1,'AVAILABLE'),(1,2,'AVAILABLE'),(1,3,'AVAILABLE'),
(1,4,'AVAILABLE'),(1,5,'AVAILABLE'),(1,6,'AVAILABLE'),
(1,7,'AVAILABLE'),(1,8,'AVAILABLE'),(1,9,'AVAILABLE'),(1,10,'AVAILABLE');

-- booking + seat update + payment
INSERT INTO bookings (user_id, show_id, status, total_amount)
VALUES (1, 1, 'CONFIRMED', 500.00);
```

```
UPDATE show_seats SET status='BOOKED', booking_id=1
WHERE show_id=1 AND seat_id IN (1,2);

INSERT INTO payments (booking_id, amount, status, gateway, external_txn_id)
VALUES (1, 500.00, 'SUCCESS', 'razorpay', 'rzp_live_abc123XYZ');
```

5. Normalization

Normal Form	Guarantee achieved
1NF — First Normal Form	All columns hold atomic, single-valued data. There are no repeating groups or arrays. For example, a booking's seats are stored as individual rows in show_seats, not as a comma-separated list inside the bookings row.
2NF — Second Normal Form	Every non-key column depends on the entire primary key. All tables use a single-column surrogate PK (id), eliminating the possibility of partial dependency. In show_seats, status and hold_expires_at depend on the full (show_id, seat_id) combination enforced as a UNIQUE key — not on either column alone.
3NF — Third Normal Form	No transitive dependencies exist. Theatre city and address live only in theatres — they are not duplicated in screens. Show base_price is a property of the show, not of the seat; per-seat pricing differences are handled via seats.seat_type, keeping show_seats free of derived data.
BCNF — Boyce–Codd Normal Form	Every determinant is a candidate key. In show_seats the UNIQUE (show_id, seat_id) constraint makes this pair a candidate key; all other columns are determined solely by it. In shows the UNIQUE (screen_id, show_date, start_time) constraint is the natural candidate key — a screen can only run one movie at a time — and all non-key attributes depend on it.

6. Concurrency & Locking Strategy

The Problem

Thousands of users may try to book the same seat simultaneously. Without protection, two transactions can both read a seat as AVAILABLE and both commit a BOOKED record — resulting in a double-booking.

Approach: Pessimistic Row-level Locking

When a user selects seats, the application opens a transaction and issues SELECT ... FOR UPDATE on the target show_seats rows. InnoDB acquires exclusive row locks. Any concurrent transaction trying to lock the same rows blocks until the first commits or rolls back.

```
START TRANSACTION;

-- Acquire exclusive locks on the requested seats
SELECT id, status
FROM show_seats
WHERE show_id = 1
AND seat_id IN (3, 4)
AND status = 'AVAILABLE'
FOR UPDATE;

-- Proceed only if rows_returned == seats_requested.
-- Transition to HELD with a 10-minute expiry.
UPDATE show_seats
SET status = 'HELD',
hold_expires_at = DATE_ADD(NOW(), INTERVAL 10 MINUTE),
booking_id = :new_booking_id
WHERE show_id = 1
AND seat_id IN (3, 4)
AND status = 'AVAILABLE';

-- If rows_affected < seats_requested → ROLLBACK
COMMIT;
```

If a second transaction arrives while the first holds the lock, it blocks. After the first commits (status = HELD), the second re-evaluates and finds 0 AVAILABLE rows — it rolls back cleanly. No double-booking possible.

Hold Expiry — Releasing Stale Holds

A background job (cron every minute, or Redis TTL keyspace notification) releases seats whose hold has timed out:

```
UPDATE show_seats
SET status = 'AVAILABLE',
hold_expires_at = NULL,
booking_id = NULL
WHERE status = 'HELD'
AND hold_expires_at < NOW();
```

INDEX idx_ss_expires (hold_expires_at) makes this sweep O(expired rows), not O(all rows), even at millions of show_seats records.

Idempotent Payment Webhooks

payments.external_txn_id carries a UNIQUE constraint. The webhook handler uses INSERT ... ON DUPLICATE KEY UPDATE, so a retried webhook from the payment gateway is safely absorbed:

```
INSERT INTO payments
(booking_id, amount, status, gateway, external_txn_id, payload)
VALUES
(:booking_id, :amount, 'SUCCESS', 'razorpay', :txn_id, :raw_json)
ON DUPLICATE KEY UPDATE
status = VALUES(status),
payload = VALUES(payload),
updated_at = NOW();
```

Index Strategy

Index	Table	Purpose
idx_shows_date_screen	shows	P2 query — filter by date + screen (theatre join)
idx_ss_show_status	show_seats	Count available seats per show efficiently
idx_ss_expires	show_seats	Hold-expiry sweep — targets only HELD rows
idx_bookings_user	bookings	Fetch a user's booking history
uq_show_seat	show_seats	Enforce one status row per (show, seat)
uq_external_txn	payments	Idempotency guarantee for payment webhooks

7. P2 — List Shows by Date & Theatre

Returns all shows running at a given theatre on a given date, including movie details, screen name, start/end times, price, and live available seat count.

```
-- P2: List all shows on a given date at a given theatre
--
-- Replace the WHERE clause values before running:
-- show_date = '2024-10-01' ← your target date
-- t.id = 1 ← your theatre_id

SELECT
m.title AS movie_title,
m.language,
m.genre,
sc.name AS screen_name,
sh.show_date,
sh.start_time,
ADDTIME(sh.start_time,
SEC_TO_TIME(m.duration_mins * 60)) AS end_time,
sh.base_price,
COUNT(ss.id) AS total_seats,
SUM(ss.status = 'AVAILABLE') AS available_seats
FROM shows sh
JOIN movies m ON m.id = sh.movie_id
JOIN screens sc ON sc.id = sh.screen_id
JOIN theatres t ON t.id = sc.theatre_id
LEFT JOIN show_seats ss ON ss.show_id = sh.id
WHERE sh.show_date = '2024-10-01'
AND t.id = 1
GROUP BY
sh.id, m.title, m.language, m.genre,
sc.name, sh.show_date, sh.start_time,
m.duration_mins, sh.base_price
ORDER BY
sh.start_time;
```

Expected Output (sample data)

movie_title	language	screen	start	end	price	avail
Stree 2	Hindi	Screen 2	11:30	13:45	200.00	10
Kalki 2898 AD	Telugu	Screen 1	10:00	13:00	250.00	8
Kalki 2898 AD	Telugu	Screen 1	14:00	17:00	300.00	10
Devara: Part 1	Telugu	Screen 3	18:00	20:46	350.00	10

Why this query is efficient

• idx_shows_date_screen (show_date, screen_id) on the shows table lets MySQL seek directly to the target date without a full scan.

• The theatre JOIN on screens.theatre_id adds only the theatre filter; theatre_id is a FK with an implicit index.

• LEFT JOIN show_seats with a conditional SUM counts available seats in one pass — no correlated subquery needed.